

Assessing and Remediating Coverage for a Given Dataset

Abolfazl Asudeh, Zhongjun Jin, H. V. Jagadish
University of Michigan
{asudeh, markjin, jag}@umich.edu

Abstract—Data analysis impacts virtually every aspect of our society today. Often, this analysis is performed on an existing dataset, possibly collected through a process that the data scientists had limited control over. The existing data analyzed may not include the complete universe, but it is expected to cover the diversity of items in the universe. Lack of adequate coverage in the dataset can result in undesirable outcomes such as biased decisions and algorithmic racism, as well as creating vulnerabilities such as opening up room for adversarial attacks.

In this paper, we assess the coverage of a given dataset over multiple categorical attributes. We first provide efficient techniques for traversing the combinatorial explosion of value combinations to *identify* any regions of attribute space not adequately covered by the data. Then, we determine the least amount of additional data that must be *obtained* to resolve this lack of adequate coverage. We confirm the value of our proposal through both theoretical analyses and comprehensive experiments on real data.

I. INTRODUCTION

In the current age of data science, it is commonplace to have a learning algorithm trained based on some dataset. This dataset could be collected prospectively, such as through a survey or a scientific experiment. In such a case, a data scientist may be able to specify requirements such as representation and coverage. However, more often than not, analyses are done with data that has been acquired independently, possibly through a process on which the data scientist has limited, or no, control. This is often called “found data” in the data science context. It is generally understood that the training dataset must be representative of the distribution from which the actual test/production data will be drawn. More recently, it has been recognized that it is not enough for the training data to be representative: it must include enough examples from less popular “categories”, if these categories are to be handled well by the trained system. Perhaps the best known story underlining the importance of this inclusion is the case of the “google gorilla” [1]. An early image recognition algorithm released by Google had not been trained on enough dark-skinned faces. When presented with an image of a dark African American, the algorithm labeled her as a “gorilla”. While Google very quickly patched the software as soon as the story broke, the question is what it could have done beforehand to avoid such a mistake in the first place.

The Google incident is not unique: there have been many other such incidents. For example, Nikon introduced a camera feature to detect whether humans in the image have their eyes open – to help avoid the all-too-common situation of the

camera-subject blinking when the flash goes off resulting in an image with eyes closed. Paradoxically for a Japanese company, their training data did not include enough East Asians, so that the software classified many (naturally narrow) Asian eyes as closed even when they were open [2]. Similarly, HP webcams were not able to detect black faces [3] due to inadequate coverage in the training data [4].

The problem becomes critical when it comes to *data-driven algorithmic decision making*. For example, judges, probation and parole officers are increasingly using algorithms to assess a criminal defendant’s likelihood to re-offend [5]. Consider a tool designed to help the judges in sentencing criminals by predicting how likely an individual is to re-offend. Such a tool can provide insightful signals for the judge and have the potential to make society safer. On the other hand, a wrong signal can have devastating effects on individuals’ lives. So it is important to make sure that the tool is trained on data that includes adequate representation of individuals similar to each criminal that will be scored by it. In § V-B, we study a real dataset of criminals used for building such a tool, published by Propublica [5]. We shall show how inadequate representation might result, for example, in predicting every widowed Hispanic female as highly likely to re-offend.

While Google’s resolution to the gorilla incident was to “ban gorillas” [6], a better solution is to ensure that the training data has enough entries in each category. Referring to the issue as “disparate predictive accuracy”, [7] also highlights that the problem often is due to the insufficient or skewed sample sizes. If the only category of interest were race, as in (most of) the examples above, there are only a handful of categories and this problem is easy. However, in general, objects can have tens of attributes of interest, all of which could potentially be used to categorize the objects. For example, survey scientists use multiple demographical variables to characterize respondents, including race, sex, age, economic status, and geographic location. Whatever be the mode of data collection for the analysis task at hand, we must ensure that there are enough entries in the dataset for each object category. Drawing inspiration from the literature on diversity [8], we refer to this concept as *coverage*.

Note that the mentioned examples, including the Google incident, are surely not sampling cases where the data scientists poorly chose the samples from a large database. Rather, they somehow collected, or acquired, a dataset, and then failed to realize the lack of coverage for dark-skinned faces.

Lack of coverage in a dataset also opens up the room for adversarial attacks [9]. The goal in an adversarial attack is to generate examples that are misclassified by a trained model. Poorly covered regions in the training data provide the adversary with opportunities to create such examples.

Our goal in this paper is two-fold. First, we would like to help the dataset users to be able to assess the coverage, as a characterization, of a given dataset, in order to understand such vulnerabilities. For example, we propose to use information about lack of coverage as a widget in the nutritional label [10] of a dataset. Once the lack of coverage has been identified, next we would like to help data owners improve coverage by identifying the smallest number of additional data points needed to hit all the “large uncovered spaces”.

Given multiple attributes, each with multiple possible values, we have a combinatorial number of possible *patterns*, as we call combinations of values for some or all attributes. Depending on the size and skew in the dataset, the coverage of the patterns will vary. Given a dataset, our first problem is to efficiently identify patterns that do not have sufficient coverage (the learned model may perform poorly in portions of the attribute space corresponding to these patterns of attribute values). It is straightforward to do this using space and time proportional to the total number of possible patterns. Often, the number of patterns with insufficient coverage may be far fewer. In this paper, we develop techniques, inspired from set enumeration [11] and association rule mining (*apriori*) [12], to make this determination efficient. We shall further discuss this and the related work in § VI.

A more interesting question for the dataset owners is what they can do about lack of coverage. Given a list of patterns with insufficient coverage, they may try to fix these, for example by acquiring additional data. In the ideal case, they will be able to acquire enough additional data to get sufficient coverage for all patterns. However, acquiring data has costs, for data collection, integration, transformation, storage, etc. Given the combinatorial number of patterns, it may just not be feasible to cover all of them in practice. Therefore, we may seek to make sure that we have adequate coverage for at least any pattern of ℓ attributes, where we call ℓ the *maximum covered level*. Alternatively, we could identify important pattern combinations by means of a *value count*, indicating how many combinations of attribute values match that pattern. Hence, our goal becomes to determine the patterns for the minimum number of items we must add to the dataset to reach a desired maximum covered level or to cover all patterns with at least a specified minimum value count. Since a single item could contribute to the coverage of multiple patterns, we shall show that this problem translates to a hitting set [13] instance. Given the combinatorial number of possible value combinations, the direct implementation of hitting set techniques can be very expensive. We present an approximate solution technique that can cheaply provide good results.

We note that not all combinations of attribute values are of interest. Some may be extremely unlikely, or even infeasible. For example, we may find few people with attribute *age*

as “teen” and attribute *education* as “graduate degree”. A human expert, with sufficient domain knowledge, is required to be in the loop for (i) identifying the attributes of interest, over which coverage is studied, (ii) setting up a *validation oracle* that identifies the value combinations that are not realistic, and (iii) identifying the uncovered patterns and the granularity of patterns that should get resolved during the coverage enhancement.

II. PRELIMINARIES

We consider a dataset $\mathcal{D}$ with d low-dimensional categorical attributes, $\mathcal{A} = \{A_1, A_2, \dots, A_d\}$. Where attributes are continuous valued or of high cardinalities, we consider using techniques such as (a) bucketization: putting similar values into the same bucket, or (b) considering the hierarchy of attributes in the data cube for reducing the cardinalities. Each tuple $t \in \mathcal{D}$ is a vector with the value of A_i being $t[i]$ for all $i = 1, \dots, d$. In addition, the dataset also contains the “label attributes” $Y = \{y_1, \dots, y_{d'}\}$ that contain the target values. The label attributes are not considered for the coverage problem. In practice, a user may be interested in studying the coverage over a subset of “attributes of interest”. In such cases, the problem is limited to those attributes. For instance, in a dataset of criminals, attributes such as *sex*, *race*, and *age* can be attributes of interest while the label attribute shows whether or not the criminal has re-offended. In the rest of the paper, we assume A_1 to A_d are the attributes of interest and simply name them as the set of attributes. The cardinality of an attribute A_i is c_i . Hence, the total number of value combinations is $\prod_{k=1}^d c_k$. For a subset of attributes $\mathcal{A}_i \subseteq \mathcal{A}$, we use the notation $c_{\mathcal{A}_i} = \prod_{A_j \in \mathcal{A}_i} c_j$ to show the number of value combinations for $\mathcal{A}_i$.

Definition 1 (Pattern). A pattern P is a vector of size d , in which $P[i]$ is either X (meaning that its value is unspecified) or is a value of attribute A_i . We name the elements with value X as non-deterministic and the others as deterministic.

An item t matches a pattern P (written as $M(t, P) = \top$), if for all i for which $P[i]$ is deterministic, $t[i]$ is equal to $P[i]$. Formally:

$$M(t, P) = \begin{cases} \top, & \forall i \in [1, d] : P[i] = X \text{ or } P[i] = t[i] \\ \perp, & \text{otherwise} \end{cases} \quad (1)$$

For example, consider the pattern $P = X1X0$ on four binary attributes A_1 to A_4 . It describes the value combinations that have the value 1 on A_2 and 0 on A_4 . Hence, for example, $t_1 = [1, 1, 0, 0]$ and $t_2 = [0, 1, 1, 0]$ match P , as their values on all deterministic elements of P (i.e., A_2 and A_4) match the ones of P . On the other hand, $t_3 = [1, 0, 1, 0]$ does not match the pattern P . That is because $P[2] = 1$ while $t_3[2] = 0$.

Using the patterns to describe the space of value combinations, we now define the coverage notion as follows:

Definition 2 (Coverage). Given a dataset $\mathcal{D}$ over d attributes with cardinalities $c = \{c_1 \dots c_d\}$, and a Pattern P based on c and d , the coverage of P is the number of items in $\mathcal{D}$ that match P . Formally: $\text{cov}(P, \mathcal{D}) = |\{t \in \mathcal{D} \mid M(t, P) = \top\}|$.

When $\mathcal{D}$ is known, we can simplify $\text{cov}(P, \mathcal{D})$ with $\text{cov}(P)$.

We would like a high enough coverage for each pattern, to make sure it is adequately represented. How high is enough is expected as an input to our problem, and is expected to be determined through statistical analyses. There is a long tradition of computing the “power” of an experiment design, to determine the subject pool size (corresponding to coverage) required to obtain statistically meaningful results. Borrowing the concept from statistics and central limit theorem, the rule of thumb suggests the number of representatives to be around 30. For example, Sudman [14] suggests that for each “minor subgroup” a minimum of 20 to 50 samples is necessary. This is what we also observed in our experiments (§ V-B). Using such, or other, techniques, we will assume that a *Coverage threshold*, τ , has been established for each pattern.

Definition 3 (Covered/Uncovered Pattern). *A pattern P is said to be covered in a dataset $\mathcal{D}$ if its coverage is greater than or equal to the specified coverage threshold: $\text{cov}(P, \mathcal{D}) \geq \tau$. Otherwise, the pattern P is said to be uncovered.*

Each pattern describes a region in the space of value combinations, constrained by its deterministic elements. We define the level of each pattern P , shown as $\ell(P)$, as the number of deterministic elements in it. Patterns with fewer deterministic elements (smaller level) are more general. For example, consider two patterns $P_1 = 1XXX$ and $P_2 = 10X1$ on four binary attributes A_1 to A_4 . $\ell(P_1) = 1$ and $\ell(P_2) = 3$. While only the value combinations 1001 and 1011 match P_2 , any value combination with value 1 on A_1 matches P_1 .

The set of value combinations that match a pattern P may be a subset of the ones that match a more general pattern P' . We say that P is *dominated* by P' (or P' dominates P). For example, the pattern $P_2 = 10X1$ is dominated by the pattern $P_1 = 1XXX$.

Definition 4 (Parent/Child Pattern). *A pattern P_1 is a parent of a pattern P_2 if P_1 can be obtained by replacing one of the deterministic elements in P_2 (say $P_2[i]$) with X . We can equivalently say that P_2 is a child of pattern P_1 .*

In general, patterns can each have multiple parents and multiple children. A pattern with all elements being non-deterministic has no parent and a pattern with all elements being deterministic has no child.

If a pattern is uncovered, all of its children, and their children, recursively, must also be uncovered. When identifying uncovered patterns, it is redundant to list all these dominated uncovered patterns: doing so just makes the output much larger, and much harder for a human to digest and address. Therefore, our goal is to identify the uncovered patterns that are not dominated by more general uncovered ones.

Definition 5 (Maximal Uncovered Pattern (MUP)). *Given a threshold τ , a pattern P is maximal uncovered, if $\text{cov}(P) < \tau$, while for any pattern P' parent of P , $\text{cov}(P') \geq \tau$.*

With these definitions, we formally state our first problem as:

Problem 1 (MUP Identification Problem). *Given a dataset $\mathcal{D}$ defined over d attributes with cardinalities c , as well as the coverage threshold τ , find all maximal uncovered patterns $\mathcal{M}$.*

While there usually are far fewer MUPs than uncovered patterns, the worst case remains bad, as we show next.

Theorem 1. *No Polynomial time algorithm can guarantee the enumeration of the set of maximal uncovered patterns.*

The proof is by construction of an example with an exponential number of MUPs. Details in the technical report [15].

Not all MUPs are problematic. For example, if some combination of attribute values is known to be infeasible, the corresponding pattern will necessarily be uncovered. A domain expert can examine a list of MUPs and identify the ones that can safely be ignored. The remaining are considered *material*.

In many situations, large uncovered regions in the dataset are more harmful than narrow uncovered regions. Following this observation, for a dataset $\mathcal{D}$, we define the *maximum covered level* as the maximum level up until which there is enough coverage in the dataset. Formally:

Definition 6 (Maximum Covered Level). *Let $\mathcal{M}$ be the material MUPs for a dataset $\mathcal{D}$. Then, the maximum covered level of $\mathcal{D}$ is the maximum level λ such that $\forall P \in \mathcal{M}, \ell(P) > \lambda$.*

In light of the above, we would like to have as large a maximum covered level as possible for a dataset.

Problem 2 (Coverage Enhancement Problem). *Given a dataset $\mathcal{D}$, its set of material MUPs $\mathcal{M}_{\mathcal{D}}$, and a positive integer number λ , determine the minimum set of additional tuples to collect such that, after the data collection, the maximum covered level of $\mathcal{D}$ is at least λ .*

We can consider variants of the coverage enhancement problem where we seek to attain some other coverage property rather than satisfy a maximum coverage level. For example, instead of the level of a pattern P , one could consider the number of value combination matching it.

Definition 7 (Value Count). *Let $\mathcal{A}_P$ be the set of corresponding attributes for non-deterministic elements of a pattern P . The value count of P is the number of value combinations matching P . That is, $c_{\mathcal{A}_P} = \prod_{A_j \in \mathcal{A}_P} c_j$.*

For example, consider the pattern $P = X1X0$ over binary attributes $\mathcal{A} = \{A_1, \dots, A_4\}$. $\mathcal{A}_P = \{A_1, A_3\}$. Hence, the number of value combinations matching P is $c_{\mathcal{A}_P} = 2 \times 2 = 4$. The coverage enhancement problem can be modified to require that every pattern P in $\mathcal{D}$ be covered if the value count of P is at least v . As further explained in § IV, the proposed solution can easily be extended for such alternative measures.

Next, in Theorem 2, we study the complexity of the coverage enhancement problem.

Theorem 2. *The Coverage Enhancement Problem is NP-hard.*

We prove the theorem using an interesting polynomial-time reduction from the *vertex cover* (VC) problem [13]. Please find the details in the technical report [15].

III. MUP IDENTIFICATION

In this section, we study Problem 1, MUP identification, and propose efficient search and pruning strategies for it.

A. Naïve

A single pass over the dataset can suffice, with one counter for each pattern. With one pass, we obtain the count for each pattern, and can determine which patterns are uncovered. We can then compare each pair of uncovered patterns, $\{P_i, P_j\}$. If P_i dominates P_j , then the latter is not maximal, and can be removed from the list of uncovered patterns discovered. After all pairs of uncovered patterns have been processed, and the ones not maximal eliminated, then the remaining uncovered patterns are the desired maximal uncovered patterns.

Suppose there are d attributes. Each element of a pattern can either be non-deterministic, or a value from the corresponding attribute. As such, there are $c_i + 1$ choices for each attribute A_i , resulting in a total of $c_A^+ = \prod_{k=1}^d (c_k + 1)$ patterns. We need one counter for each pattern, or a total space of $O(c_A^+)$. The time taken to find all uncovered patterns is $O(n \times c_A^+)$, where there are n tuples in the dataset. Let the total number of uncovered patterns found be u . Then an additional $O(u^2)$ time is required to find the maximal uncovered patterns from among these. Thus, the total time required is $O(n c_A^+ + u^2)$. While the additional time due to the second term will usually be smaller than the first term, we note that u could be as large as c_A^+ , and is usually much larger than the number of maximal uncovered patterns. As a toy example, consider Example 1.

Example 1. Consider a dataset $\mathcal{D}$ with binary attributes A_1, A_2 , and A_3 , containing the tuples $t_1 : 010, t_2 : 001, t_3 : 000, t_4 : 011$, and $t_5 : 001$. Let the coverage threshold be $\tau = 1$.

The dataset in Example 1 has one MUP 1XX. In addition to the MUP, the other 8 uncovered patterns (dominated by the only MUP) are 1X0, 1X1, 10X, 11X, 100, 101, 110, and 111.

B. Pattern Graph

In the naïve algorithm, we computed all uncovered patterns, only to eliminate those that were not maximal. It would appear that we could do less work if we could exploit relationships between patterns. Specifically, patterns have parent/child relationships, as discussed in § II. We can represent relationships between patterns by means of a pattern graph, and use this data structure to find better algorithms.

Definition 8 (Pattern Graph). Let $\mathcal{P}$ be the set of all possible patterns defined over d attributes with cardinalities c . Pattern graph of $\mathcal{P}$ is the graph $G(V, E)$ where $V = \mathcal{P}$. There is an edge between every pair of nodes P and P' that have a parent-child relationship. Every edge is between two nodes at adjacent levels, the parent node being one level smaller than the child node.

Figure 1 shows the pattern graph for Example 1. The value of $P[i]$ for a node in the pattern graph is either X or one of the values the corresponding attribute can take. Hence, the total number of nodes in a pattern graph defined over d attributes

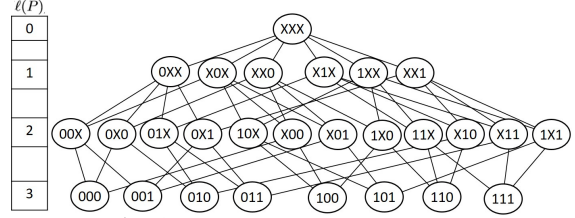


Fig. 1: The pattern graph for three binary attributes

is $c_A^+ = \prod_{k=1}^d (c_k + 1)$. For instance, the pattern graph in Figure 1 contains $(2 + 1)^3 = 27$ nodes. Any pattern graph has only one node $XXX \dots X$ at level 0. In Figure 1, the patterns at level 1 are directly connected to XXX as those are its children. The pattern $0XX$, the left-most node at level 1, is connected to the patterns $00X$, $0X0$, $01X$, and $0X1$ at level 2. That is because, those have exactly one less X and their value for the first attribute is 0. As explained in § II, the number of value combinations matching P (with non-deterministic attributes $\mathcal{A}_P$) is $c_{\mathcal{A}_P} = \prod_{A_j \in \mathcal{A}_P} c_j$. The number of non-deterministic attributes of a pattern P with level $\ell(P)$ is $d - \ell(P)$. For example, in Figure 1, every pattern at level 1 contains $3 - 1 = 2$ non-deterministic elements. Since the cardinality of all attributes is $c_i = 2$, the number of value combinations matching each pattern at level 1 is $2 \times 2 = 4$. Hence, in general, the patterns with smaller levels are more general, i.e., more value combinations match them. Each node at level ℓ contains $d - \ell$ non-deterministic elements. There are $\binom{d}{\ell}$ such combinations in all. The deterministic elements can take any value in the cardinality of the corresponding attribute. Hence, for the special case where all attributes have the same cardinality $c_i = c$, total number of nodes at level ℓ are $\binom{d}{\ell} c^\ell$. For example, in Figure 1, there are $\binom{3}{1} 2^1 = 6$ nodes at level 1 and $\binom{3}{2} 2^2 = 12$ nodes at level 2. The node of a pattern P in the pattern graph has $\sum_{A_i \in \mathcal{A}_P} c_i$ edges to the nodes at level $\ell(P) + 1$. If all attributes have equal cardinalities of $c_i = c$, each node at level ℓ has $c(d - \ell)$ edges to nodes at level $\ell + 1$. Hence, the total number of edges in such a graph is:

$$\sum_{\ell=0}^{d-1} c(d - \ell) \binom{d}{\ell} c^\ell = c \times d \times (c + 1)^{d-1}$$

This is confirmed in Figure 1, where there are totally 54 edges.

C. PATTERN-BREAKER (the top-down algorithm) and PATTERN-COMBINER (the bottom-up algorithm)

In this section we propose the top-down and bottom-up algorithms PATTERN-BREAKER and PATTERN-COMBINER, that lead to the design of our final MUP identification algorithm, DEEPDIVER. Due to the space limitations, we provide a sketch of these algorithms and refer the reader to the technical report [15] for further details.

PATTERN-BREAKER starts from the general patterns at the top of the pattern graph and moves down by breaking them down to more specific ones. It uses the “monotonicity” property of coverage to prune some parts of the pattern graph. That is, if a pattern P is uncovered, all of its children and descendants (the nodes at level greater than $\ell(P)$ that have a

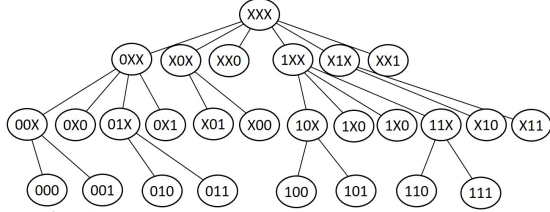


Fig. 2: Tree transformation for Figure 1 based on Rule 1.

path to it) are also uncovered. Also, none of those children and descendants can be a MUP, even if it has a parent that is covered. Hence, this subgraph of the pattern graph can immediately get pruned. In the BFS traversal of the pattern graph, the algorithm enforces Rule 1, states in the following, to ensure that each MUP candidate is generated exactly once.

Rule 1. A node P with the coverage more than the threshold τ , generates the candidate nodes at level $\ell(P) + 1$ by replacing the non-deterministic elements in the right-hand side of its right-most deterministic element with an attribute value.

Theorem 3. Enforcing Rule 1 guarantees that each MUP candidate is generated exactly once.

By enforcing Rule 1, the pattern graph is transformed to a *tree*. For example, Figure 2 shows the corresponding tree (generated by following Rule 1) for the pattern graph of Figure 1.

The other algorithm PATTERN-COMBINER, performs a bottom-up traversal of the pattern graph. It uses an observation that the coverage of a node at level ℓ of the pattern graph can be computed using the coverage values of its children at level $\ell + 1$. The algorithm also uses the monotonicity of the coverage to prevent the complete traversal of the graph. That is, the coverage of a node is not less than the coverage of any of its children. It starts from the most specific patterns, i.e., the patterns at level d of the graph, computes the coverage of each by passing over the data once. The algorithm then, keeps combining the uncovered patterns at each level to get the coverage of the candidate nodes at level $\ell - 1$. The uncovered nodes at level ℓ that all of their parents at level $\ell - 1$ are covered are MUPs. PATTERN-COMBINER transforms the pattern graph to a *forest*, in order to make sure every node is generated once. Rule 2 guarantees the transformation. Figure 3 shows the transformation of Figure 1 to a forest, based on Rule 2.

Rule 2. A node P with the coverage less than the threshold τ , generates the candidate nodes at level $\ell(P) - 1$ by replacing the deterministic elements with value 0 in the right-hand side of its right-most non-deterministic element with X .¹

Theorem 4. Enforcing Rule 2 guarantees each MUP candidate is generated exactly once.

D. DEEPDIVER: Fast search space pruner

PATTERN-BREAKER traverses over the covered regions of the pattern graph before it visits the uncovered patterns.

¹Note that this rule is not specific to the binary attributes. All we require is that one of the values of each attribute is mapped to 0.

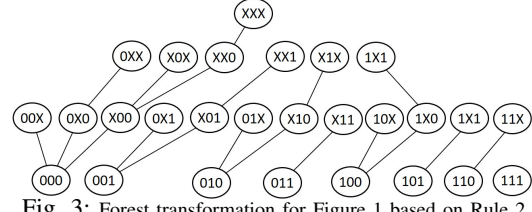


Fig. 3: Forest transformation for Figure 1 based on Rule 2.

Therefore, it does not perform well when a large portion of the pattern graph is covered. Conversely, PATTERN-COMBINER traverses over the uncovered nodes first; so it will not perform well if most of the nodes in the graph are uncovered. When most MUPs are in the middle of the graph, both algorithms do poorly because they have to traverse about half of the graph. In this subsection, we propose DEEPDIVER, an algorithm that tries to quickly identify some MUPs and use them to prune the search space.

The monotonicity property creates an opportunity to prune the search space: *none of the ancestors or descendants of a given MUP can be MUP*. PATTERN-BREAKER tends to traverse level by level over the covered regions of the pattern graph before it visits the uncovered patterns. As a result, the moment it reaches out to a MUP, it already has visited its ancestors and does not take the advantage of pruning the nodes dominating the MUPs. PATTERN-COMBINER, on the other hand, starts off in the uncovered regions; initially, the nodes being visited early are at the bottom of the pattern graph. It gradually moves up level by level until it hits the MUPs. Therefore, when the MUPs are discovered the descendants have already been visited and, as a result, PATTERN-COMBINER does not take the advantage of pruning the nodes dominated by MUPs.

With the above observations, we propose DEEPDIVER, a search algorithm that tends to quickly find MUPs, and use them to limit the search space by pruning the nodes both dominating and dominated by the discovered MUPs. Since each MUP is the child of a covered node, instead of scanning through the covered/uncovered patterns level by level, DEEPDIVER takes a path down to find an uncovered node.

Initially, DEEPDIVER(Algorithm 1), following a DFS strategy, takes a path down until it reaches into an uncovered region in the graph. However, the discovered uncovered pattern is not necessarily a MUP, as some of its other parents (other than its generator) might also be uncovered. For instance in Example 1, assume that in the first iteration, the algorithm take the path $XXX \rightarrow X0X \rightarrow 10X$. The nodes XXX and $X0X$ are covered, but $10X$ is not. Still the uncovered node $10X$ is not a MUP as it has the uncovered parent $1XX$. Therefore, after finding an uncovered node, DEEPDIVER changes direction and starts moving up to find a MUP. To do so, it checks the parents of the current node to see if any of them are uncovered. If there exists such a parent, it moves to the parent and continues until it finds a MUP. Upon discovering a MUP, DEEPDIVER prunes all of its ancestors and descendants, and continues the search for other MUPs in the regions that are still not pruned. The algorithm stops when all of the nodes in the pattern graph are

Algorithm 1 DEEPDIVER**Input:** Dataset $\mathcal{D}$ with d attributes having cardinalities c and threshold τ **Output:** Maximal uncovered patterns $\mathcal{M}$

```

1: Let  $S$  = an empty stack
2: push  $X \cdots X$  to  $S$ 
3: while  $S$  is not empty do
4:    $P$  = pop a node from  $S$ 
5:    $uncoveredFlag$  = a flag indicating if  $P$  is uncovered
6:   if  $P$  is dominated by  $\mathcal{M}$  then
7:     continue
8:   else if  $P$  dominates  $\mathcal{M}$  then
9:      $uncoveredFlag$  =  $true$ 
10:  else
11:     $cnt$  =  $cov(P, \mathcal{D})$ 
12:     $uncoveredFlag$  =  $cnt < \tau$ 
13:  end if
14:  if  $uncoveredFlag$  is  $true$  then
15:    Let  $S'$  = an empty stack
16:    while  $S'$  is not empty do
17:       $P'$  = pop a node from  $S'$ 
18:       $P'$  = generates parent nodes of  $P'$  by replacing one deterministic cell with  $X$ .
19:      for  $P'' \in \mathcal{P}'$  do
20:         $cnt'$  =  $cov(P'', \mathcal{D})$ 
21:        if  $cnt' < \tau$  then push  $P''$  to  $S'$ ; break
22:      end for
23:      add  $P$  to  $\mathcal{M}$ 
24:    end while
25:  else
26:     $Q$  = generate nodes on  $P$  and  $c$  based on Rule 1
27:    push  $Q$  to  $S$ 
28:  end if
29: end while
30: return  $\mathcal{M}$ 

```

pruned.

Here we extend the notion pattern dominance to MUP dominance, as follows:

Definition 9 (MUP Dominance). *Given a pattern P and a set of MUPs $\mathcal{M}$, P is dominated by $\mathcal{M}$, if there exists a pattern $P' \in \mathcal{M}$ such that P is dominated by P' . Similarly, P dominates $\mathcal{M}$, if there exists a pattern $P' \in \mathcal{M}$ such that P dominates P' .*

Based on the Definition 9, a node being dominated by MUPs is not a MUP. DEEPDIVER uses this property to limit the search space by pruning all descendants of the MUPs. Similarly, the nodes that dominate MUPs are out of the search space. We use inverted indices for efficiently checking MUP dominance. See the technical report [15] for details.

IV. COVERAGE ENHANCEMENT

Every MUP represents a part of the value combinations space for which there are not enough observations in the dataset. For example, consider a dataset defined over three ternary attributes A_1 , A_2 , and A_3 , in which MUPs are XX1, 0XX, and 20X. Figure 4 shows the matches for the patterns XX1, 0XX, and 20X, as the red, green, and blue cubes, respectively. The more general MUPs show larger uncovered regions in the data. For example in Figure 4, the cube of the more general patterns XX1 and 0XX contain 9 combinations, whereas the one for the pattern 20X contains 3 combinations. While we may be willing to leave some small regions

uncovered, we would like to cover at least the large ones. For example, not having enough representatives in a dataset for single black males over the age of sixty may be less of a problem than not having enough black males. Figure 5 shows the distribution of the levels of the MUPs for a real experiment on our AirBnB dataset (c.f. § V) with $n = 1000$ items and $d = 13$ attributes, while $\tau = 50$. There are several thousand MUPs in this setting. This indicates the high expense of covering them all. However, as the distribution has a bell-curve shape, while most MUPs appear at levels 5 and 6, there is only one MUP at level one and less than forty in level two.

Data acquisition is usually costly. If the data are obtained from some third party, there may be direct monetary payment. If the data are directly collected, there may be a data collection cost. In all cases, there is a cost to cleaning, storing, and indexing the data. To minimize these costs, we would like to acquire as few additional tuples as possible to meet our coverage objective.

Before discussing further technical details, we would like to emphasize the necessity of *human-in-the-loop* after the MUP discovery. Not all the MUPs that are discovered are meaningful and some of them may even be invalid. Therefore after the MUP discovery, a domain expert should evaluate and mark out the MUPs that are not problematic. In addition, we require the expert to set up a *validation oracle* as a set of rules that identifies if a value combination is semantically correct or not. For example, any value combination that contains {gender=Male, isPregnant=True} is semantically incorrect.

Definition 10 (Validation Rule). *A validation rule is a set of pairs $\{\langle A_i, V_i \rangle, \dots\}$, where A_i is an attribute and V_i is a set of values for A_i . Given a pattern P and a validation rule R we say P satisfies R , if $\forall \langle A_i, V_i \rangle \in R: P[i] \in V_i$.*

Definition 11 (Validation Oracle). *A validation oracle contains a collection of validation of rules. Given a pattern P the oracle returns true if P satisfies none of its validation rules. It returns false otherwise.*

The human expert sets up the validation oracle by identifying the collection of validation rules. Later on, in this section, we call the validation oracle to enforce the rules that result in the semantic appropriateness (validity) of the output of the coverage enhancement algorithm.

As formally defined in § II as Problem 2, for a given value λ our objective is to collect the minimum number of additional tuples such that after the data collection the maximum covered level of $\mathcal{D}$ is at least λ . It is not enough to cover the MUPs with levels $\ell \leq \lambda$, we must cover all uncovered patterns (not necessarily maximal) with level $\ell(P) = \lambda$. We use M_λ to refer to the set of uncovered patterns at level λ . Finding this set is not difficult: details in [15].

We take Example 2 as a running example in this section.

Example 2. *Consider a dataset $\mathcal{D}$ with 5 attributes A_1 to A_5 where A_2 and A_3 are ternary attributes while the other attributes are binary. Suppose the maximal uncovered patterns are as shown in Figure 7.*

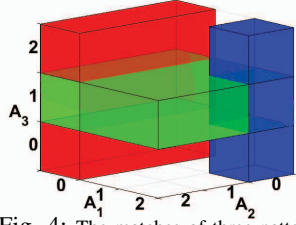


Fig. 4: The matches of three patterns XX1: green, OXX: red, and 20X: blue in a data set with ternary attributes A_1 , A_2 , and A_3 .

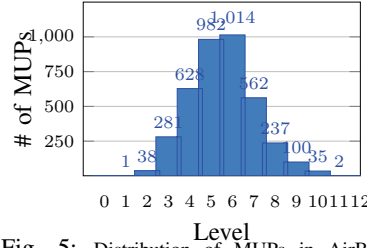


Fig. 5: Distribution of MUPs in Airbnb dataset for $n = 1000$ items and $d = 13$ attributes, while $\tau = 50$.

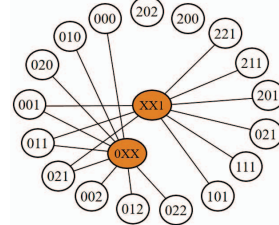


Fig. 6: The bipartite graph for $\lambda = 1$ for Figure 4.

P_1	XX01X
P_2	1X20X
P_3	XXXX1
P_4	02XXXX
P_5	XX11X
P_6	111XX
P_7	X020X

Fig. 7: MUPs of Example 2.

Let λ be 2. Uncovered patterns of Example 2 with level 2, i.e. M_λ , are P_1 to P_6 . Our objective is to cover all these patterns.

If we use an alternative problem formulation, the set of patterns to cover may be different. For example, if we wish to cover all patterns with value count of at least v , we must enumerate uncovered patterns that meet this value count criterion. Once this (straightforward) enumeration is completed, thereafter the alternative problem formulation can be solved in exactly the same way. A potential naive idea may be to acquire enough additional tuples separately for each pattern we are required to cover. However, this “solution” acquires much more than the minimum data required, because each tuple may contribute to the coverage of multiple patterns. What we need is to choose tuples carefully to find the minimum number needed to cover all the uncovered patterns of interest. This problem translates to a hitting set [13] instance.

A. Transformation to hitting set

Hitting Set Problem: Given a set $\mathcal{U}$ of elements and a collection $\mathcal{S}$ of non-empty subsets of $\mathcal{U}$, the objective is to find the smallest subset of elements $C \subseteq \mathcal{U}$ such that $\forall S \in \mathcal{S}, \exists e \in C$ where $e \in S$.

The transformation is as follows:

- $\mathcal{U}$: The set of possible value combinations translates to the universe of items $\mathcal{U}$.
- $\mathcal{S}$: Each uncovered pattern in M_λ is the representative of the set of value combinations matching it. Hence, $\mathcal{S}$ is the collection of sets represented by the uncovered patterns.

This can be viewed as a bipartite graph with the value combinations in the first part and the uncovered patterns in the second part. There is an edge between a combination and a pattern if the combination matches the pattern. The objective is to select the minimum number of nodes in the first part that hit all the patterns in the second part. Figure 6 shows the bipartite graph for $\lambda = 1$ in Figure 4.

While the hitting set problem is NP-complete, the greedy approach guarantees a logarithmic approximation ratio for it [13]. At every iteration, the greedy approximation algorithm selects the item (value combination) that hits the maximum number of un-hit sets (patterns). It continues until all the sets get hit. In Figure 6, for instance, a run of the greedy algorithm picks 001 as it hits both patterns and then stops.

At every iteration, the algorithm needs to find the value combination that hits the maximum number of un-hit patterns. This is inefficient due to the exponentially large number of the

	P_1	P_2	P_3	P_4	P_5	P_6
$A_1 = 0$	1	0	1	1	1	0
$A_1 = 1$	1	1	1	0	1	1
$A_2 = 0$	1	1	1	0	1	0
$A_2 = 1$	1	1	1	0	1	1
$A_2 = 2$	1	1	1	1	1	0

Fig. 8: The inverted indices for values of attributes A_1 and A_2 in Example 2.

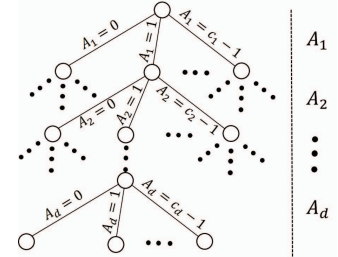


Fig. 9: The tree data structure used for computing the number of patterns a value combination hits.

value combinations and potentially exponential number of the patterns to hit. Hence, in the following, we develop an efficient implementation of the greedy algorithm.

B. Efficient implementation of the greedy algorithm

Consider the set of patterns we want to hit by the value combinations. We use inverted indices to keep track of the uncovered patterns. The i -th element of each pattern is either a value of A_i or is non-deterministic. For each attribute value v_j of A_i , we create an inverted index to point to the patterns with either a non-deterministic element or an element with value v_j in the i -th position. We use this to filter out the patterns that do not match a value combination with value v_j on A_i . For example, Figure 8 shows the indices for the values of A_1 and A_2 for P_1 to P_6 in Example 2. The first row shows the inverted index for $A_1 = 0$. All columns of the row, except P_2 and P_6 are 1; that is because a value combination having $A_1 = 0$ will not match P_2 or P_6 , but still can match other patterns. Having the inverted indices for the attribute values, we use a tree data structure and design a greedy threshold-based algorithm to find the value combination that hits the maximum number of remaining patterns.

Consider a full tree data structure (Figure 9) with depth d (the number of attributes). Let m be the number of patterns that we want to hit. The root has c_1 children, each showing a value for attribute A_1 . Similarly, every node at level i has c_i children, each representing an attribute value of A_i . For instance, in Example 2, the depth of the tree is $d = 5$; the root node has two children standing for $A_1 = 0$ and $A_1 = 1$. Since $c_2 = 3$, each of these two nodes will have three children $A_2 = 0$, $A_2 = 1$, and $A_2 = 2$. Each path in the tree from the root to a leaf shows a value combination. In [15], we

show how to use the inverted indices for finding the number of remaining patterns a value combination hits.

Using this structure, we design the threshold-based algorithm GREEDY (pseudo-code in [15]) that uses the hit-count of its best-known value combination to prune the tree. The algorithm traverses through the tree data structure in a DFS manner and, starting from the root, it calls the validation oracle before generating each child of a node, to make sure it is semantically meaningful. As a result, it will output only the value combination that are valid. The algorithm computes the bit vector of valid children of a node by applying the binary AND operation between the current bit vector and the inverted index of each of its children. The algorithm uses the best-known value combination as a lower-bound threshold to prune the tree. If the children of the current node are leaf nodes and the best of them hits more patterns than the best-known value combination, the best option gets updated. For the other nodes, the algorithm prioritizes the children of the current node based on the number of 1's in their bit-vectors. Then, starting from the one with the max count, until the number of 1's in the bit-vectors is more than the best-known hit count, it recursively checks if it can find a better value combination in the subtrees of the children. The algorithm keeps collecting the value combinations that hit the maximum number of remained patterns until all of the patterns in M_λ get hit. As an implementation note, when the algorithm selects a value combination, it takes the intersection of the patterns it hits to find a more general pattern that any of its matching value combinations hit the same set of patterns. It provides more freedom to the user in the data collection.

V. EXPERIMENTAL EVALUATION

We conducted comprehensive experiments on real data to both validate our proposal and to study the efficiency of the proposed algorithms in practice.

A. Experimental Setup

Datasets. Three real datasets were used for the experiments:

- **COMPAS**²: ProPublica is a nonprofit organization that produces investigative journalism. They collected and published the COMPAS dataset as part of their investigation into racial bias in criminal risk assessment. The dataset contains demographics, recidivism scores, and criminal offense information for 6,889 individuals. We used the attributes *sex* (0: male and 1: female), *age* (0: under 20, 1: between 20 and 39, 2: between 40 and 59, and 3: above 60), *race* (0: African-American, 1: Caucasian, 2: Hispanic, and 3: other races), and *marital status* (0: single, 1: married, 2: separated, 3: widowed, 4: significant other, 5: divorced, and 6: unknown) for studying the coverage.
- **AirBnB**³: AirBnB is a popular online peer to peer travel marketplace that provides a framework for people to lease or rent short-term lodging. We use a collection of the information of approximately 2 million *real* properties enlisted

in AirBnB. The website provides 41 attributes for each property, out of which 36 are boolean attributes, such as TV, internet, washer, and dryer.

- **BlueNile**⁴: Blue Nile is the largest online diamond retailer globally. We collected its catalog containing 116,300 diamonds at the time of access. The dataset has 7 categorical attributes for the diamonds, namely shape, cut, color, clarity, polish, symmetry, and fluorescence with cardinalities 10, 4, 7, 8, 3, 3, and 5, respectively.

Hardware and Platform. The experiments were conducted on a Linux machine with a 3.8 GHz Intel Xeon processor and 64 GB memory. The algorithms were implemented in Java.

Experiments plan. We want to study coverage in real data. Are there indeed uncovered patterns? Are these likely to cause errors in prediction or analysis? We also want to study the performance of the proposed algorithms, both for MUP identification and for coverage enhancement. We studied both these sets of questions on all three datasets. In the interests of space, we report only the most salient results. In particular, we focus on the COMPAS dataset for the first set of questions, since the negative consequences of lack of coverage are potentially more severe than for other datasets where the impact may be limited to errors in analytical results. We focus on the AirBnB dataset for the performance questions, since this is the largest of the three datasets. Since attributes of AirBnB are binary, we supplement with the BlueNile dataset to highlight situations where its much higher cardinality of attribute values matters.

B. Validation

1) *Issues with Coverage in Real Data:* Consider four demographic attributes *sex*, *age*, *race*, and *marital status*, as the attributes of interest in the COMPAS dataset. We investigate the lack of coverage in this dataset with regard to these four attributes to show the risks of using it for important tasks such as assessing a criminals' likelihood to re-offend and sentencing them accordingly. Setting the threshold to 10, all the single attribute values contain more instances than the threshold. Still, there totally are 65 MUPs in this dataset, out of which 19 are in level $\ell = 2$, 23 in level $\ell = 3$, and 23 in level $\ell = 4$. Besides other MUPs, the existence of 19 level two MUPs in the dataset emphasizes the potential of bad predictions for large spaces in the data cube. To highlight one example, the MUP XX23 shows the lack of coverage for *widowed Hispanics*. The dataset contains only two instances matching this pattern and interestingly both of them have offended multiple times. In the absence of enough representatives for the minority subgroup, the trained model, will likely generalize, not sticking to the couple of examples it has seen for the minority subgroup. However, the generalization becomes problematic when the "behavior" in the subgroup is different and the generalization is misleading. This means that the model *may* not do a good job in modeling the behaviour of minority sub-groups. Of course, we use the

²www.propublica.org/dataset/compas-recidivism-risk-score-data-and-analysis

³www.airbnb.com

⁴www.bluenile.com/diamond-search?

MUP identification to raise a signal for these lack of coverage cases. Whether or not it is problematic, needs the human expert in the loop. Lack of coverage in this dataset shows the risk of using it for predicting the behavior of individuals for under-represented groups; it, therefore, questions the decisions made based on such predictions. To show case an effect of the lack of coverage, next we use this dataset for training a classifier.

2) *Lack of coverage's effect*: After showing the lack of coverage in the COMPAS dataset, next we conduct an experiment to show its effect on accuracy of a prediction task. Using the scikit-learn package (version 0.20) on Python, we trained a *decision tree* as the classifier, while using `sex`, `age`, `race`, and `marital status` as the observation attributes. Using the attribute `prior-count`, we created the binary label attribute that shows if a criminal has re-offended. First, using the cross-validation, we observed that the trained model has acceptable accuracy and f1 measures of 0.76 and 0.7 over a random test set. Relying on these numbers, a data scientist may consider using this model for predicting the behaviour of criminals. However, in this experiment we show that these measures does not necessarily show the good performance for the minorities. We focus on the minority class of Hispanic Females (HF), as there are only 100 of those in the dataset. We chose this group, specifically because we were limited to the records in the dataset, and were not able to collect additional data points. Hispanic Females was (i) a minority subgroup small enough that removing its instances would not noticeably change the size of the training data, while (ii) there were “enough” tuples of this group in the dataset (100 tuples) that we could show case the impact of additional data collection. We considered a randomized set of 20 (out of the 100) HF as the test set for studying the prediction over this group. Since we not only wanted to study the effect of the lack of coverage, but also the coverage enhancement, we created 5 training sets (using the remaining 80 HF criminals), containing $\{0, 20, 40, 60, 80\}$ HF plus all other records not in this demographic. We used these datasets for training the classifier and calculated its accuracy and f1 measure for predicting our test set of 20 HF. Figure 10 shows the results. The x-axis shows the datasets, while the left and right y-axes show the accuracy and f1 measure for each setting. Collecting the additional data points should result in more accurate for the under-represented groups, as those are to provide a better understanding of those, while not having a major impact in the overall accuracy. This is confirmed in the figure, as the overall accuracy remained on 76% in all settings. We also observed that overall f1-measure did not change from 0.7. First, one can see that the dataset that does not have any HF, has an unacceptable performance for this class, as its accuracy is less than 50%. The next observation is that the accuracy and f1 measures improve as the lack of coverage is resolved by adding more HF to the training data. The reduction in the slope of the accuracy curve around 40 suggests that it can be a good choice for the coverage threshold. Interestingly, this is aligned with the central limit theorem's rule of thumb of 30. In a similar experiment, we considered two other minority subgroups, (1. Female - Other Races (FO) and 2. Male - Other

Races (MO)) for which there existed at least 20 records in the dataset that we could consider as the test data. Removing the records of these demographics from the training data, the accuracy of the model was 39% for FO and 59% for MO. The accuracy different between the two groups shows the higher similarity in the “behaviour” of MO to other records in the training data.

3) *Coverage Enhancement Quality*: In previous experiment, we showed the quality of coverage enhancement in the sense that it increases the model performance for the under-represented groups, while not impacting the overall performance of the model. In this experiment, we show the role of human-in-the-loop by setting up the validation oracle and identifying the MUPs to be covered. Enforcing the rules of validation oracle while expanding the tree data structure used by the coverage enhancement algorithm GREEDY, the semantic appropriateness (validity) of the output of the coverage enhancement algorithm is guaranteed. We consider the MUPs discovered in § V-B1 while targeting to satisfy the coverage level of 2. In the validation oracle, we rule out (a) the combinations with marital status being unknown and (b) the age group below 20 being not single. Coverage enhancement suggests to collect $\{\text{over } 60, \text{other races, widowed}\}$, $\{\text{between } 20 \text{ and } 40, \text{Hispanic, widowed}\}$, $\{\text{over } 60, \text{significant other}\}$, $\{\text{other races, divorced}\}$, and $\{\text{other races, widowed}\}$.

C. Performance Evaluation

We evaluate the performance of (i) the three MUP identification algorithms PATTERN-BREAKER, PATTERN-COMBINER, and DEEPDIVER, as well as (ii) the coverage enhancement algorithm. The Naïve algorithm for MUP identification (§ III-A) did not finish for any of the settings within the time limit. Therefore, we did not include it in the results. For the coverage enhancement problem, we compare the GREEDY algorithm (§ IV-B) with the direct implementation of the hitting set's approximation algorithm (naïve). We use our largest dataset, i.e. AirBnB, as the default and test the algorithms' performances under various coverage threshold (τ) on both AirBnB and BlueNile. We varied the number of attributes (d) and the size of the dataset (n) on our largest dataset, that is AirBnB. In addition to the proposed algorithms, for MUP identification, we also consider comparing with APRIORI, the following adaptation of apriori algorithm [12]: we consider each $\langle \text{attribute}, \text{value} \rangle$ as an item and find the frequent item-sets. For each such item-set we find its parents (the item-sets that include the item-set and one more item). For each such parent, if all of its children are frequent, we find the corresponding pattern and add it to the set of MUPs. We understand that because the items are considered independently in the frequent item-set mining, not all item-set represent a valid pattern. For instance, consider the items $I_1 = \langle A_1, 0 \rangle$ $I_2 = \langle A_1, 1 \rangle$. Then the item-set $\{I_1, I_2\}$ does not represent a valid pattern. Furthermore, this algorithm considers a much larger search space (lattice) to explore, compared to the pattern graph our algorithms explore. For instance, consider a case where there are 10 attributes, each with cardinality 5. The

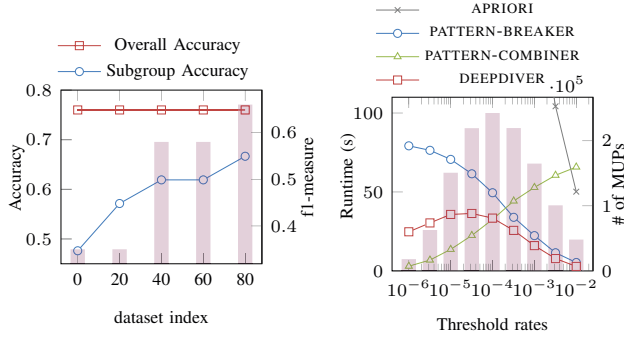


Fig. 10: The effect of lack of coverage on classification: accuracy, f1 measure

Fig. 11: AirBnB: MUP identification, varying threshold ($n = 1M$, $d = 15$)

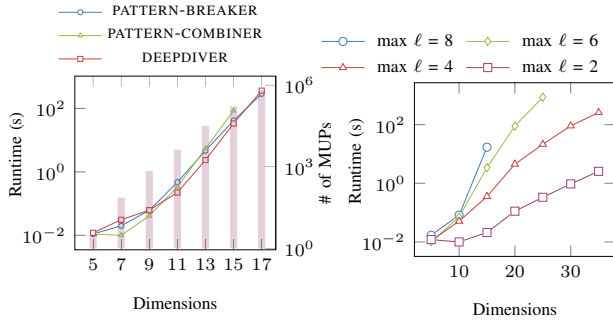


Fig. 14: AirBnB: MUP identification, varying dimension ($n = 1M$, $\tau = 0.1\%$)

Fig. 15: MUPs identification with various dimensions using DEEPDIVER (AirBnB, $n = 1M$, $\tau = 0.1\%$)

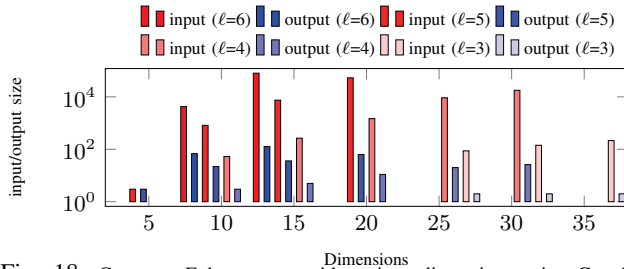


Fig. 18: Coverage Enhancement with various dimensions using Greedy (AirBnB, $n = 1M$, $\tau = 0.1\%$) – input/output sizes of the pattern graph is $(5+1)^{10}$, around 60 million nodes, whereas after considering each attribute-value as an item, the size of the defined lattice is $2^{5 \times 10}$, around 10^{15} .

1) *MUP identification - varying threshold*: For AirBnB, we varied the coverage threshold from 0.0001% (most patterns are covered) to 1% (most patterns are uncovered). The dataset size was set to one million, and the number of attributes was set to 15. For BlueNile, we had 7 attributes and 116,300 records. We varied coverage from 0.001% (threshold = 1) to 1%.

Results. The runtimes are shown in Figure 11 (AirBnB) and Figure 12 (BlueNile). The x-axis denotes the different threshold rate values. The left-y-axis is the runtime in seconds, while the right axis and the bars show the output size (number of MUPs). In addition, Figure 11 also contains the results for APRIORI, the adaptation of the apriori algorithm for discovering the MUPs. As explained in § V-C, this algorithm suffers from multiple facts that makes it unsuitable for MUP

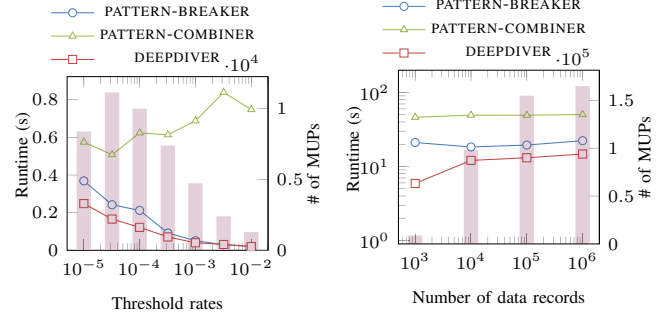


Fig. 12: BlueNile: MUP identification, varying threshold ($n = 116,300$, $d = 7$)

Fig. 13: AirBnB: MUP identification, varying data size ($\tau = 0.1\%$, $d = 15$)

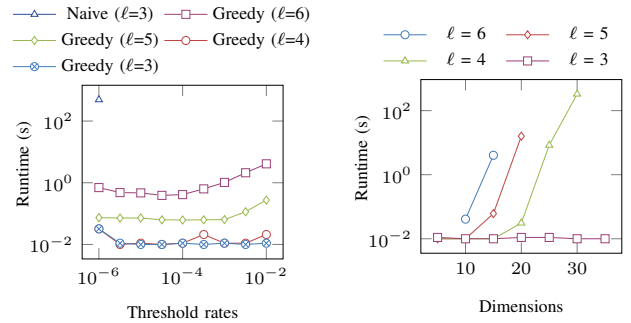


Fig. 16: Coverage Enhancement with various thresholds (AirBnB, $n = 1M$, $d = 13$)

Fig. 17: Coverage Enhancement with various dimensions using Greedy (AirBnB, $n = 1M$, $\tau = 0.1\%$)

discovery. First, the lattice data structure it has to explore can be extensively larger than the pattern graph. Second, it needs to generate the parents of the frequent item-sets to find the infrequent item-sets that all of their children are frequent. Finally, not all the discovered item-sets represent valid MUPs. This is confirmed in this experiment where it only finished for one settings in less than 100 seconds. For instance for threshold of 0.001% it took 516 sec. to finish. As expected, we observed the same behaviour in other experiments as well. Hence, in the rest of experiments, we only focus on evaluating the algorithms we proposed in this paper. When the threshold increases, larger regions in the space become uncovered and more general MUPs with smaller levels appear in the results. This is the reason for the drop in the runtime of PATTERN-BREAKER in Figure 11 and Figure 12. Recall that PATTERN-BREAKER is a top-down search algorithm, and generally returns faster when the MUPs are higher in the pattern graph (having small levels). In contrast, PATTERN-COMBINER's runtime increases as the PATTERN-COMBINER is a bottom-up search algorithm and, hence, terminates faster when the MUPs are low in the pattern graph (when the space is mostly covered) as shown in Figure 11 and Figure 12. In tests with AirBnB, these two algorithms have similar speeds when the threshold is around 0.01%, in which case, most MUPs appear in the middle of the graph. Meanwhile, Figure 11 also shows that DEEPDIVER is as fast, if not faster, as the other two algorithms in all situations. This suggests that the efficiency

of DEEPDIVER is more robust to the actual data coverage status. As for BlueNile, Figure 12 also suggests DEEPDIVER is the best in all cases, whereas PATTERN-COMBINER is always slower. Still the gap between PATTERN-COMBINER with the two other algorithms is larger. The high cardinality of the attributes in BN is the key to this behavior. In this situation, the width of the pattern graph quickly increases. The lowest level (level 7) of the pattern graph in this case has more than 100K nodes, whereas for 7 binary attributes, it is 128. Therefore, due to the significant width of the graph in the bottom-level, PATTERN-COMBINER (the bottom-up algorithm) loses its efficiency.

2) *MUP identification - varying data size:* Setting the number of attributes to 15 and threshold to 1%, we evaluated the three MUP identification algorithms on data samples of various sizes from 10K to 1M and measured the runtime.

Results. Figure 13 shows the runtime plots. The x-axis denotes the size of test dataset; the left-y-axis denotes the runtime in seconds and the right-y-axis (and the bars) show the number of MUPs. All three algorithms had running time only slightly impacted by data set size, taking less than 100 seconds in all settings. The effort is driven more by the number of patterns, which is independent of data set size. The PATTERN-COMBINER algorithm checks the actual dataset only for the bottom layer of the pattern graph and so the data set size has no effect on most of its computation. PATTERN-BREAKER and DEEPDIVER need to check the data for computing the coverage of the intermediate nodes, so data set size does matter. However, the use of inverted indices limits the impact.

3) *MUP identification - varying data dimensions:* Similarly, we evaluate the scalability of the proposed algorithm as the number of attributes (d) increases. With a dataset size of one million records and the threshold set at 1%, we measured the overall runtime of all three algorithms with the dataset projected down to between 5 and 17 dimensions.

Results. In Figure 14, the x-axis denotes the number of attributes, while the left-y-axis and right-y-axis (the bars) denote the runtime in seconds and the output size, respectively. The size of the pattern graph increases exponentially with the number of attributes. The number of MUPs and the algorithm running times also increase exponentially. Still, all algorithms managed to finish in a reasonable time (under two minutes) for up to 17 attributes.

As the number of attributes increases, the number of MUPs increases exponentially, but those become the combination of more attributes. While the MUPs with fewer are harmful and important to discover, the MUPs with more attributes are too specific, and hence, less interesting. For example, while lack of coverage for Hispanic males in a dataset is an important fact to discover, not having enough married Hispanic males under the age of 20 is less harmful. Limiting the exploration level to a certain number, allows the MUP identification algorithms to scale for datasets with tens of attributes and still finding the risky MUPs. We evaluated this by limiting the MUP discovery level in Figure 15 while using DEEPDIVER for the

identification. As observed in the figure, the algorithm was able to quickly find MUPs of up to level 2 (the MUPs that are the combinations of one or two attributes) for even 35 attributes in around 10 sec.

4) *Coverage enhancement - varying threshold:* Recall that the objective is to identify the minimum additional data to collect, such that after the data collection the maximum coverage level is not less than λ , i.e. there are no uncovered patterns on or above a given level λ . Setting the number of items to 1M in the AirBnB dataset and number of attributes to 13, we vary the threshold rate from 10^{-6} to 0.01 while choosing different maximum coverage levels from 3 to 6.

Results. Figure 16 represents the experiment results. The x-axis shows threshold and the y-axis provides the runtime in seconds. First, the single blue triangular tick mark in the top-left of the plot shows the only setting for which the naïve algorithm finished within the time limit. GREEDY, on the other hand, finished in a few seconds for all settings. The next observation is that, as expected, the runtime of the GREEDY algorithm increases by the level; that is because it needs to collect more data points to ensure that there is no uncovered pattern on or above level λ , i.e., $\forall P \in \mathcal{M} : \ell(P) \geq \lambda$. Also, as the threshold rate increases the MUPs move to the top of the pattern graph. Therefore, more regions in the space become uncovered and more data points are required to guarantee the given maximum coverage level. As a result, the algorithm's runtime increases by the threshold.

5) *Coverage enhancement - varying data dimensions:* Lastly, we study the effect of the number of attributes on the performance of GREEDY, as well as input and output sizes. Using the AirBnB dataset, while setting the number of items to 1M and the threshold to 1%, we vary the number of attributes from 5 to 35, and the max. coverage level from 3 to 6.

Results. Figure 17 shows the runtime of the algorithm, while Figure 18 provides information about the input and output sizes. Here, by the input size, we refer to the number of uncovered patterns (to cover) at the given level λ while the output size is the number of additional data points to collect. First, as explained above, increasing the maximum coverage level increases the runtime of the algorithm, as the output size increases. This is also reflected in Figure 18, as for a fixed number of attributes, both the input and output size increase in orders of magnitude. Similarly, increasing the number of attributes increases the size of the pattern graph exponentially, and also does the algorithm runtime (Figure 17) and the output size (Figure 18). Still, recall that lack of coverage for the patterns that are the combination of a few attribute values (having smaller levels) is more harmful than the ones in the form of the combination of several attribute values. Looking at Figure 17, while solving the coverage enhancement problem for larger levels takes more time, the algorithm has a reasonable performance for resolving the lack of coverage for smaller values of maximum coverage level. Finally, in Figure 18, applying the greedy approximation algorithm, the output sizes are significantly smaller than the input sizes for

each setting. That is because every value combination in the output hits multiple uncovered patterns in the input.

VI. RELATED WORK

Diversity, as a general term for capturing the quality of a collection of items on the variety of its constituent elements [8], is an important issue in a wide range of contexts, including social science [16], political science [17], information retrieval [18], and big data environments and data ethics [19], [8]. Facility dispersion problems [20] tend to disperse a set of points such that the minimum or average distance between the pair of points is maximized. Also, techniques such as determinantal point process (DPP) have been used for diverse sampling [21], [22]. A recent work [23] considers diversity as the entropy over one discrete low-cardinality attribute. Our definition of coverage can be seen as a generalization of this, defined over combinations of multiple attributes.

The rich body of work on sampling, especially in the database community, aims to draw samples from a large database [24], [25]. Our goal in this paper is to ensure that a given dataset (often called as “found data”) is appropriate for use in a data science task. The dataset could be collected independently, through a process on which the data scientist have limited, or no, control. This is different from sampling.

Technically speaking, there are similarities between the algorithms provided in this paper and the classical powerset lattice and combinatorial set enumeration problems [11], such as data cube modeling [26], frequent item-sets and association rule mining [12], data profiling [27], and data cleaning [28]. While such work, and the algorithms such as apriori, traverse over the powerset lattice, our problem is modeled as the traversal over the pattern graph which has a different structure (and properties) compared to a powerset lattice. Hence, those techniques cannot be directly applied here. We provided some rules for traversing the pattern graph that are inspired from the set enumeration tree [11], one-to-all broadcast in a hypercube [29], and lattice traversal heuristic proposed in [27]. In § IV, we modeled the data collection problem as a hitting set instance (an equivalent of the set cover problem). Further details about this fundamental problem can be found in references such as [13], [30].

VII. FINAL REMARKS

In this paper, we studied lack of coverage as a risk to using a dataset for analysis. Lack of coverage in the dataset may cause errors in outcomes, including algorithmic racism. Defining the coverage over multiple categorical attributes, we developed techniques for identifying the spots not properly covered by data to help the dataset users; we also proposed techniques to help the dataset owners resolve the coverage issues by additional data collection. Comprehensive experiments over real datasets demonstrated the validity of our proposal.

Following ideas such as [14], in MUP identification problem, we considered a fixed threshold across different value combinations, representing “minor subgroups”. We consider

further investigations on identifying threshold value and minor subgroups, as well as other alternatives for future work.

VIII. ACKNOWLEDGEMENTS

This work was supported by NSF Grant No. 1741022. We are grateful to the University of Toronto, Department of Computer Science, and Dr. Nick Koudas for the AirBnB dataset.

REFERENCES

- [1] M. Mulshine. A major flaw in google’s algorithm allegedly tagged two black people’s faces with the word ‘gorillas’. *Business Insider*, 2015.
- [2] Adam Rose. Are face-detection cameras racist? *Time Business*, 2010.
- [3] Mallory Simon. HP looking into claim webcams can’t see black people. *CNN*, 2009.
- [4] Tess Townsend. Most engineers are white and so are the faces they use to train software. *Recode*, 2017.
- [5] Julia Angwin, Jeff Larson, Surya Mattu, and Lauren Kirchner. Machine bias: Risk assessments in criminal sentencing. *ProPublica*, 5/23/2016.
- [6] Alex Hern. Google’s solution to accidental algorithmic racism: ban gorillas. *The Guardian*, 2018.
- [7] Irene Chen, Fredrik D Johansson, and David Sontag. Why is my classifier discriminatory? In *NeurIPS*, 2018.
- [8] Marina Drosou, HV Jagadish, Evaggelia Pitoura, and Julia Stoyanovich. Diversity in big data: A review. *Big data*, 5(2), 2017.
- [9] Battista Biggio, Igino Corona, Davide Maiorca, Blaine Nelson, Nedim Šrđić, Pavel Laskov, Giorgio Giacinto, and Fabio Roli. Evasion attacks against machine learning at test time. In *ECML PKDD*, 2013.
- [10] Ke Yang, Julia Stoyanovich, Abolfazl Asudeh, Bill Howe, HV Jagadish, and Jerome Miklau. A nutritional label for rankings. In *SIGMOD*, 2018.
- [11] Ron Rymon. Search through systematic set enumeration. Technical report, University of Pennsylvania, 1992.
- [12] Rakesh Agrawal and Ramakrishnan Srikant. Fast algorithms for mining association rules. In *VLDB*, 1994.
- [13] Vijay V Vazirani. *Approximation algorithms*. Springer Science & Business Media, 2013.
- [14] Seymour Sudman. Applied sampling. *Academic Press New York*, 1976.
- [15] A. Asudeh, Z. Jin, and H. V. Jagadish. Assessing and remedying coverage for a given dataset. *CoRR*, abs/1810.06742, 2018.
- [16] Edward H Simpson. Measurement of diversity. *Nature*, 163(4148), 1949.
- [17] James Surowiecki. *The wisdom of crowds*. Anchor, 2005.
- [18] Rakesh Agrawal, Sreenivas Gollapudi, Alan Halverson, and Samuel Jeong. Diversifying search results. In *WSDM*, pages 5–14. ACM, 2009.
- [19] Solon Barocas and Andrew D Selbst. Big data’s disparate impact. *Cal. L. Rev.*, 104:671, 2016.
- [20] SS Ravi, Daniel J Rosenkrantz, and Giri Kumar Tayi. Facility dispersion problems: Heuristics and special cases. In *WADS*. Springer, 1991.
- [21] Alex Kulesza, Ben Taskar, et al. Determinantal point processes for machine learning. *Foundations and Trends in ML*, 5(2–3), 2012.
- [22] N. Anari, Sh. O. Gharan, and A. Rezaei. Monte carlo markov chain algorithms for sampling strongly rayleigh distributions and determinantal point processes. In *COLT*, pages 103–115, 2016.
- [23] L Elisa Celis, Amit Deshpande, Tarun Kathuria, and Nisheeth K Vishnoi. How to be fair and diverse? *CoRR*, abs/1610.07183, 2016.
- [24] Frank Olken and Doron Rotem. Random sampling from databases: a survey. *Statistics and Computing*, 5(1):25–42, 1995.
- [25] Graham Cormode, Minos Garofalakis, Peter J Haas, Chris Jermaine, et al. Synopses for massive data: Samples, histograms, wavelets, sketches. *Foundations and Trends in Databases*, 4(1–3):1–294, 2011.
- [26] Venky Harinarayan, Anand Rajaraman, and Jeffrey D Ullman. Implementing data cubes efficiently. In *SIGMOD*, 1996.
- [27] A. Heise, J.A. Quiáné-Ruiz, Z. Abedjan, A. Jentzsch, and F. Naumann. Scalable discovery of unique column combinations. *PVLDB*, 2013.
- [28] J. He, E. Veltri, D. Santoro, G. Li, G. Mecca, P. Papotti, and N. Tang. Interactive and deterministic data cleaning. In *SIGMOD*, 2016.
- [29] D.P. Bertsekas, C. Özveren, G.D. Stamoulis, P. Tseng, and J.N. Tsitsiklis. Optimal communication algorithms for hypercubes. *JPDC*, 11(4), 1991.
- [30] Dorit S Hochbaum. *Approximation algorithms for NP-hard problems*. PWS Publishing Co., 1996.